



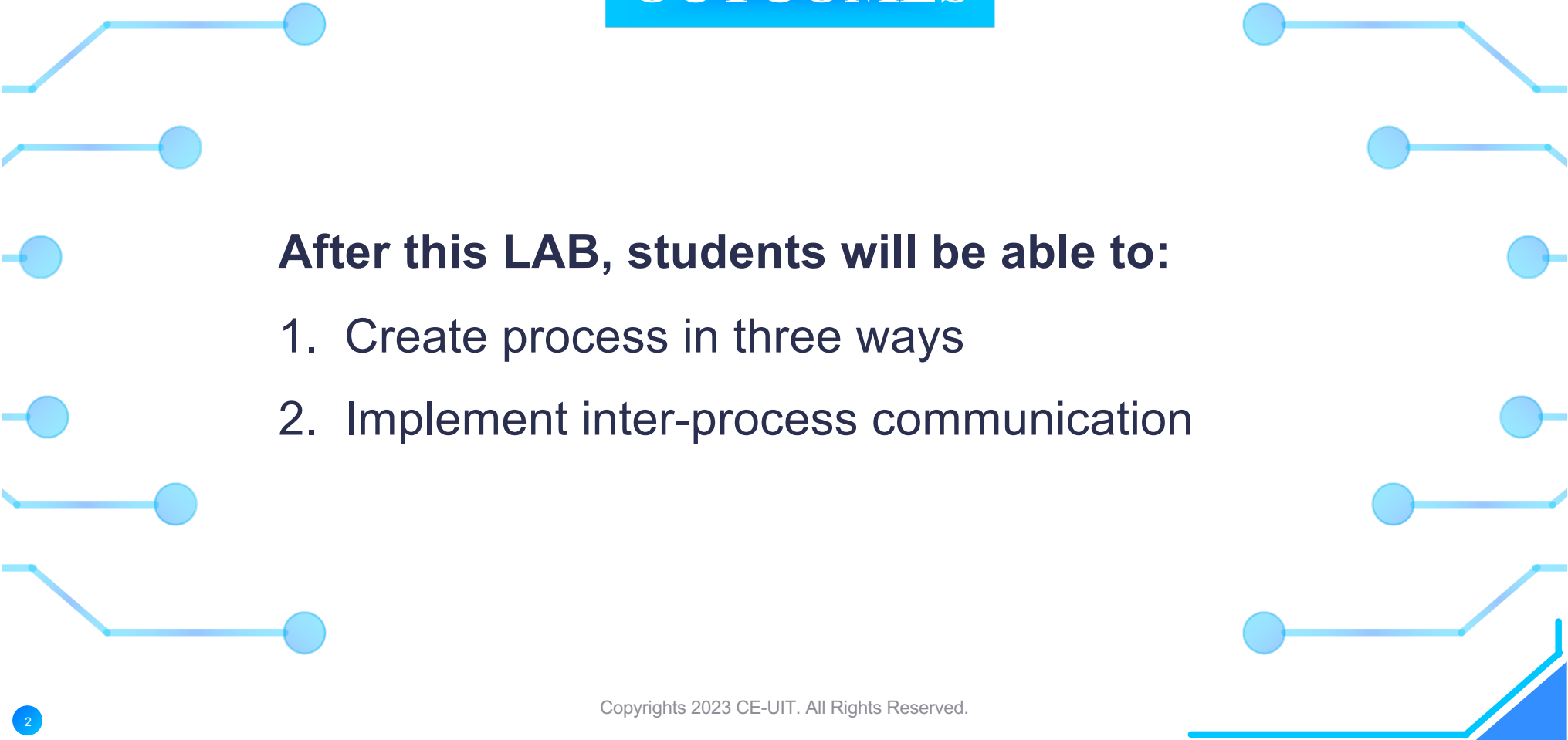
OPERATING SYSTEM

LAB 04 | WORKING WITH PROCESSES IN C

Let's learn how can we create and work with processes in C



OUTCOMES



After this LAB, students will be able to:

1. Create process in three ways
2. Implement inter-process communication



CONTENS

PREPARE

1. Introduction to Google Colab



CONTENS

PRACTICE

1. Process creation
2. Inter-Process Communication



INTRODUCTION TO GOOGLE COLAB

Prepare

00.



0. Introduction to Google Colab

What is Google Colab?

- A cloud-based Jupyter notebook environment
- Allows you to write and execute Python code and shell command
- Provides free access to GPUs and TPUs for accelerated computing





0. Introduction to Google Colab

Key features:

Collaborative Environment

Share notebooks and collaborate in real-time with others

Pre-installed Libraries

Comes with many popular libraries like TensorFlow, Keras, PyTorch, and more

Cloud Storage Integration

Easily save and load files from Google Drive

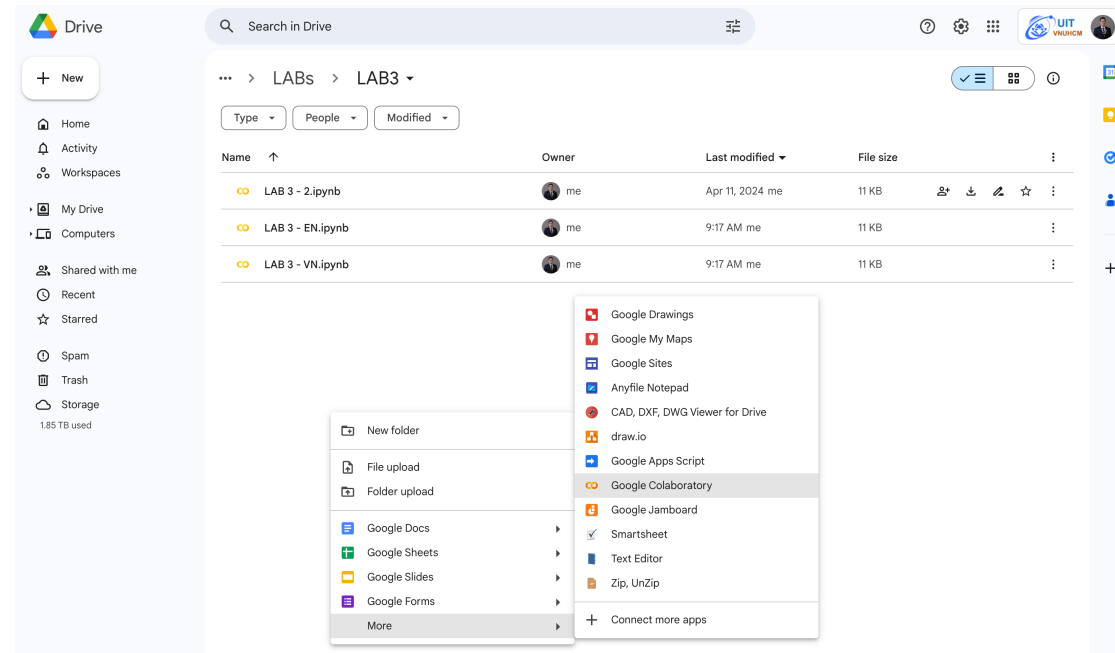


0. Introduction to Google Colab

Getting started

Access: colab.research.google.com

Or you can directly
create a Google
Colab Notebook in
your Google Drive





0. Introduction to Google Colab

Sample

LAB 3 - EN.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

✓ RAM
Disk

Gemini

STUDENT INFORMATION

Full name: Hoang-Loc Tran

Student ID: 13520462

Class: CSBU108.021

LAB 03

Write and Combine C program

```
1 %writefile sample.cpp
2
3 #include <stdio.h>
4 #define NAME "LUCAS"
5
6 int main()
7 {
8     printf("Hello world, %s!", NAME); // In dong chu "Hello world"
9
10    return 0;
11 }
```

Overwriting sample.cpp

Tạo và đọc nội dung của file tạm thời, được tạo ra sau bước Pre-processing

```
1 !g++ sample.cpp -o sample
2 !./sample
```

Hello world, LUCAS!

Markdown text

Code

Output

Output



0. Introduction to Google Colab

Change runtime

Change runtime type

Runtime type

Python 3

Hardware accelerator

- ☒ CPU ☐ T4 GPU ☐ A100 GPU ☐ L4 GPU
- ☐ TPU v2

Want access to premium GPUs? [Purchase additional compute units](#)

Cancel

Save

You can change your runtime type by choosing

Menu Runtime → Runtime type



0. Introduction to Google Colab

GUI

Start connecting to server

The screenshot shows the Google Colab interface. At the top, there's a header with the Colab logo, the notebook name 'Sample Notebook.ipynb', and a star icon. Below this is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A status bar indicates 'All changes saved'. On the left is a sidebar with icons for file explorer, search, and a key icon. The main area shows a code cell with the text '1 Start coding or generate with AI.' and a play button icon. Annotations with arrows point to various parts: a purple arrow points from the 'Connect' button in the top right to the text 'Start connecting to server'; a purple arrow points from the play button icon to the text 'Press to run cell'; a purple arrow points from the text '1 Start coding or generate with AI.' to the text 'Write cell'; a purple bracket groups 'Write cell' and 'Python code, Shell command, Magic cell' under the label 'Markdown text'.

Sample Notebook.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

1 Start coding or generate with AI.

Press to run cell

Write cell

Markdown text

Python code, Shell command, Magic cell

Connect Gemini



0.1. Markdown text

What is markdown?

- A lightweight markup language
- Used for formatting text with plain text syntax
- Commonly used in Jupyter notebooks for documentation



0.1. Markdown text

Basic markdown syntax

Markdown	How to use	Output	Notes
#	# Header 1 ## Header 2	Header 1 Header 2	Headers are collapsible
*	*Emphasis*	<i>Emphasis</i>	
**	**Bold**	Bold	



0.1. Markdown text

Basic markdown syntax

Markdown	How to use	Output	Notes
<code>`</code>	This is <code>`Inline code`</code>	This is <code>Inline code</code>	
<code>```\n```\n</code>	<code>```\n# Code block\nprint("Hello world!")\n```\n</code>	<code># Code block\nprint ("Hello world")</code>	



0.1. Markdown text

Basic markdown syntax

Markdown	How to use	Output	Notes
1.	1. Item 1	1. Item	Ordered list
*	* Item 1	• Item 1	Unordered list
[Link Text](URL)	[Google](https://www.google.com)	Google	Embedded link



0.1. Markdown text

Basic markdown syntax

Markdown	How to use	Output	Notes
<code>![Alt Text](Image URL)</code>	<code>![Logo](https://www.uit.edu.vn/sites/vi/files/resize/images/Logos/Logo_UIT_Web_Transparent-218x261.png)</code>	 UIT TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN	



You can also use this toolbar to create markdown



0.2. Code

Python Code

✓ [7] 1 print ("Hello world")
0s

⇒ Hello world

Shell Command

✓ [8] 1 !ls /content
0s

⇒ sample sample.cpp sample_data

Magic cell

✓
0s



```
1 %%writefile hello.cpp
2 #include <stdio.h>
3
4 int main()
5 {
6     printf("Hello world!");
7     return 0;
8 }
```

⇒ Writing hello.cpp



0.2. Code

Shell Command

- You can execute shell command in Google Colab which help you interact with server
- **Remember to add exclamation mark (!) before the command**

```
!chmod 755 script.sh
```

```
✓ [8] 1 !ls /content  
0s  
📁 sample sample.cpp sample_data
```



0.2. Code

Magic cell

- In Google Colab, a “**magic cell**” refers to the use of IPython magic commands. These are special commands prefixed with one or two percent signs (%) or (%%) that provide a shorthand way to perform various tasks
- In this subject, we only use %%**writefile** to create a CPP source code.



0.2. Code

Magic cell

✓
0s



```
1 %%writefile hello.cpp
2 #include <stdio.h>
3
4 int main()
5 {
6     printf("Hello world!");
7     return 0;
8 }
```



Writing hello.cpp

This magic cell creates
hello.cpp file





0.2. Code

Compile and Run C++ program



Combine C source

```
!gcc source.c -o output_file
```

Combine C++ source

```
!g++ source.cpp -o output_file
```

If you use your machine instead of Google Colab, remember to remove (!) in the command



0.2. Code

Compile and Run C++ program

Run the program

```
!path_to_output_file
```

Example

```
!./output
```

```
!/content/output
```

If you use your machine instead of Google Colab, remember to remove (!) in the command



PROCESS CREATION

1.1. Create process with fork()

01.



1.1. Create process with fork()

Classwork: 01

Read and implement section

Section: 1.1. Create process with fork()

Answer below questions:

- a. Describe how **fork()** works?
- b. What are characteristics of processes created by **fork()**?
 - Relation with original process? PID? Source code? How can we define/assign jobs for them?



PROCESS CREATION

1.2. Create process with `exec()`

01.



1.2. Create process with exec()

- The *exec family* of functions **replaces** the current process image with a new process image. It **does not** create a new process; instead, it **transforms** the existing process into a new process.



1.2. Create process with exec()

Classwork: 02

Read and implement section

Section: 1.2. Create process with exec()

Answer below questions:

- a. Explain the syntax of `execlp()` function?
- b. What are characteristics of processes created by `exec()`?
 - Relation with original process? PID? Source code? How can we define/assign jobs for them?



PROCESS CREATION

1.3. Create process with `system()`

01.



1.3. Create process with system()

- **system()** executes a shell command specified in the form of a string.
 - It forks a child process
 - Runs the shell command
 - Waits for the command to finish
 - and then returns.



1.3. Create process with system()

Classwork: 03

Read and implement section

Section: 1.3. Create process with system()

Answer below questions:

- a. Explain the syntax of `system()` function?
- b. What are characteristics of processes created by `system()`?
 - Relation with original process? PID? Source code? How can we define/assign jobs for them?



1. Process creation

Classwork: 04

Draw a table and compare the similarities and differences between creating processes using `fork()`, `exec()`, and `system()`?
Based on this comparison, explain in which cases each function should be used?



INTER-PROCESS COMMUNICATION

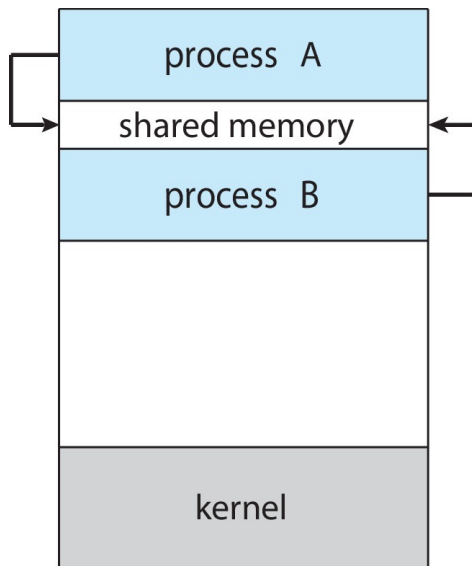
02.



2. Inter-Process Communication

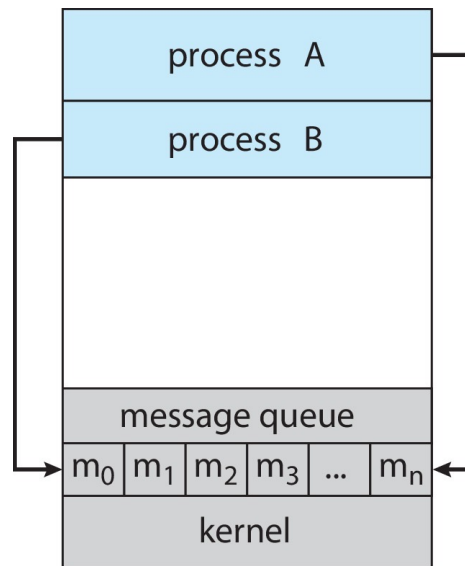
There are 02 mechanisms

Shared memory



(a)

Message passing



(b)



2.1. Message passing

- In Unix-like operating systems, **pipes** are used for inter-process communication (IPC), allowing one process to send data to another process.



2.1. Message passing

Procedure:

1

Create a Pipe

Use the `pipe()` system call to create a pipe. This call returns two file descriptors: one for reading and one for writing

System call `pipe()`

`pipefd[0]`: The file descriptor for reading from the pipe

`pipefd[1]`: The file descriptor for writing to the pipe



2.1. Message passing

Procedure:

2

Fork a Child Process

Use the pipe() system call to create a pipe. This call returns two file descriptors: one for reading and one for writing

3

Close Unnecessary Ends

Example: In the parent process, close the reading end of the pipe. In the child process, close the writing end of the pipe



2.1. Message passing

Procedure:

4

Write and Read

Example: The parent process writes data to the pipe, and the child process reads from the pipe

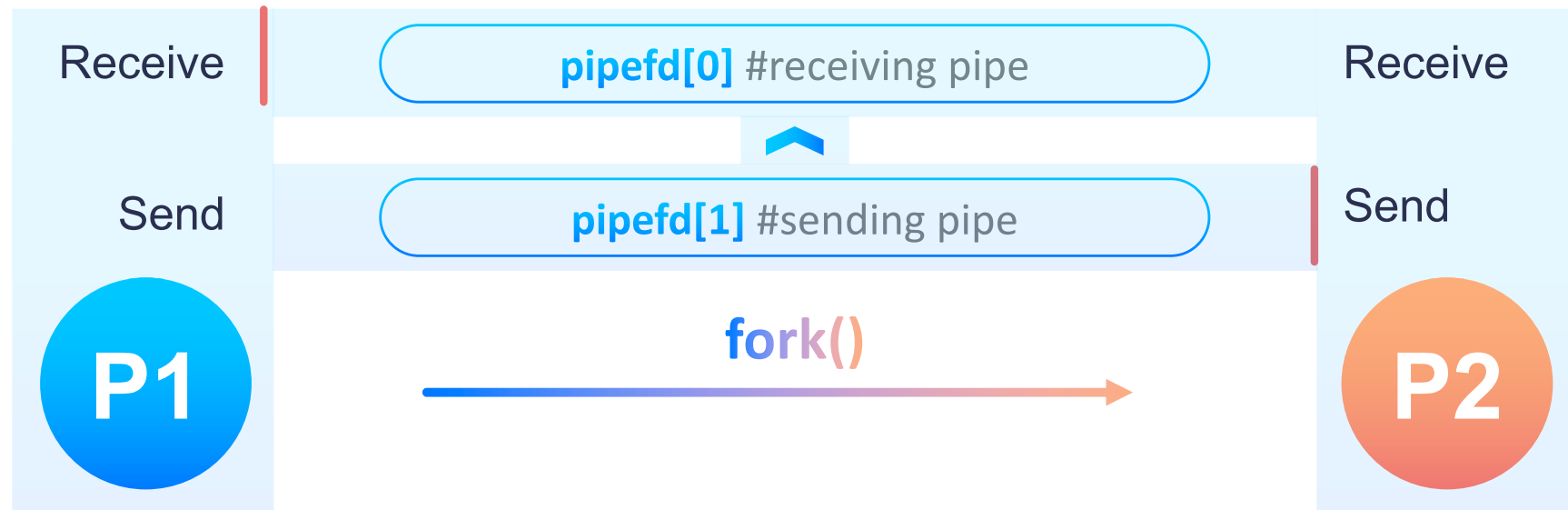
5

Close the Pipe

After communication is done, close the pipe in both processes



2.2. Example using pipe



Section: 2. Inter-Process Communication: Message Passing



EXERCISES

This section will be updated after in-class session

03.



3. Exercises

Exercise: 01

Write a program `time.cpp` that measures the execution time of a shell command. The program will be executed with the syntax `./time <command>` where `<command>` is the shell command you want to measure the execution time for. Example:

```
$ ./time ls
time.c
time
Execution time: 0.25422
```



3. Exercises

Exercise: 02

Design a file-copying program named `filecopy.cpp` using ordinary pipes. This program will be passed two parameters: the name of the file to be copied and the name of the destination file. The program will then create an ordinary pipe and write the contents of the file to be copied to the pipe. The child process will read this file from the pipe and write it to the destination file. For example, if we invoke the program as follows:

```
./filecopy input.txt copy.txt
```

The file `input.txt` will be written to the pipe. The child process will read the contents of this file and write it to the destination file `copy.txt`.